

Day 04: Microservices Communication & API Gateway

By the end of this session, learners will be able to:

- Understand **inter-service communication** in microservices architecture
- Differentiate between **synchronous and asynchronous communication models**
- Implement **REST-based service interaction**
- Use **WebClient and RestTemplate** for service calls
- Design and configure an **API Gateway using Spring Cloud Gateway**
- Apply **centralized authentication and JWT validation**
- Configure **routing and global filters**
- Understand **Spring Security fundamentals**

Microservices Communication: Conceptual Foundation

In a microservices architecture, services operate independently but must **communicate to fulfill business workflows**.

Example Scenario (CineVerse)

- User Service → authenticates user
- Movie Service → fetches movie data
- Booking Service → processes seat booking

To complete a booking:

- Booking Service must interact with both User and Movie services

Communication Requirement

- Data exchange
- Service coordination
- Workflow completion

Academic Insight

Inter-service communication **introduces network latency, failure points, and consistency challenges**, which must be addressed through proper design.

Synchronous vs Asynchronous Communication

In a monolithic application, components communicate via simple method calls. In Microservices, components are distributed across different servers, necessitating Inter-Process Communication (IPC).

Synchronous Communication

- Request → Wait → Response
- Blocking operation

Example

- Booking Service calls Movie Service and waits for response.

Advantages

- Simple to implement
- Immediate response

Limitations

- High latency
- Tight coupling
- Failure propagation

Asynchronous Communication

- Request sent → No waiting
- Response handled later

Example

Booking event sent to queue → Notification service processes it later

Advantages

- Loose coupling
- Better scalability
- Improved resilience

Comparison

Feature	Synchronous	Asynchronous
Coupling	Tight Coupling (Both must be online)	Loose Coupling (Via Message Broker)
Latency	Higher (blocked threads)	Lower (non-blocking)
Response	Immediate	Delayed
Scalability	Limited	High
Use Case	Fetching User Profile, Payment Auth	Sending Emails, Order Processing
Protocols	REST, Grpc	RabbitMQ, Apache Kafka, AWS SQS

Design Insight

Use synchronous communication **for critical real-time operations**, and asynchronous for **background processing**.

REST-Based Communication

REST (Representational State Transfer) is the gold standard for synchronous communication. It leverages HTTP methods (GET, POST, PUT, DELETE) to manipulate resources via standardized endpoints.

Principles

- Stateless
- Resource-based
- HTTP methods

Why REST?

- Lightweight
- Platform-independent
- Widely supported

Implementation: RestTemplate vs WebClient

Example (RestTemplate – The Blocking Approach)

RestTemplate is synchronous. The execution thread is blocked until the response is received from the external service.

```
@Service
public class OrderService {
    @Autowired
    private RestTemplate restTemplate;

    public InventoryResponse checkInventory(Long productId) {
        String url = "http://inventory-service/inventory/" + productId;
        // Execution stops here until inventory-service responds
        return restTemplate.getForObject(url, InventoryResponse.class);
    }
}
```

Analysis:

If the Inventory service experiences high latency, the Order service's threads will be held captive, potentially leading to "Thread Exhaustion" and a total system crash.

Limitations of RestTemplate

- Blocking
- Less efficient for reactive systems

Example (WebClient – The Reactive Approach)

WebClient is part of Spring WebFlux. It is non-blocking and asynchronous, allowing a single thread to handle multiple concurrent requests.

```
@Service
public class OrderService {
    private final WebClient webClient;

    public OrderService(WebClient.Builder webClientBuilder) {
        this.webClient = webClientBuilder.baseUrl("http://inventory-service").build();
    }
}
```

```

public Mono<InventoryResponse> checkInventory(Long productId) {
    return this.webClient.get()
        .uri("/inventory/{id}", productId)
        .retrieve()
        .bodyToMono(InventoryResponse.class);
    // Returns a Mono (Promise); thread is released immediately
}
}

```

Analysis:

WebClient does not wait for the response. Instead, it registers a callback. This is critical for building high-throughput, scalable microservices.

Advantages of Web Client

- Non-blocking
- Reactive programming support
- Better performance under load

Academic Insight

WebClient aligns with Reactive Systems principles, enabling better resource utilization.

API Gateway: Architectural Role

API Gateway acts as a single entry point for all client requests.

Core Objectives of a Gateway:

- Encapsulation: Clients interact with one URL instead of managing multiple service IPs.
- Cross-Cutting Concerns: Centralizes authentication, logging, and rate limiting.
- Security: Hides the internal network structure from the public internet.

Responsibilities

- Request routing
- Authentication validation
- Load balancing
- Rate limiting

Architecture Flow

Client → API Gateway → Microservices

Insight

Gateway abstracts internal service structure, improving **security and maintainability**.

API Gateway Implementation

Technology Used: Spring Cloud Gateway

Spring Cloud Gateway:

SCG is built on Spring WebFlux and consists of three main components:

- Route: The destination mapping (ID URI).
- Predicate: A condition that must be met for the route to be chosen (e.g., path match).
- Filter: Logic applied to the request or response before/after routing.

Core Features

- Route mapping
- Filters
- Security integration

Route Configuration

Routes define how requests are forwarded.

Example Configuration

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://ORDER-SERVICE # lb = Load Balanced via Eureka
          predicates:
            - Path=/api/orders/**
          filters:
            - AddRequestHeader=X-Gateway-Token, GatewayPass

        - id: user-service-route
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
```

Explanation

- /movies/** → forwarded to Movie Service
- Gateway acts as router

Technical Breakdown:

- lb://: This instructs the gateway to use the Service Registry (Eureka) to find the available instances of the service.
- predicates: If the incoming request path starts with /api/orders/, the gateway triggers this specific route.
- filters: In this example, we add a custom security header to the request before it ever reaches the backend microservice.

CENTRALIZED AUTHENTICATION & SECURITY

In a distributed system, traditional Session-based authentication is avoided because it requires "Session Stickiness" or a shared session database.

The Solution: JWT (JSON Web Token)

JWTs are stateless. All user information is encoded into a cryptographically signed token.

- Header: Algorithm and Token type.
- Payload: User claims (UserId, Roles, Expiration).
- Signature: A hash created using a secret key to prevent tampering.

Global Filters

Filters intercept requests and responses.

Types

- Pre-filters (before request)
- Post-filters (after response)

Use Cases

- Logging
- Authentication
- Header modification

Example

@Component

```
public class LoggingFilter implements GlobalFilter {  
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {  
        System.out.println("Request intercepted");  
        return chain.filter(exchange);  
    }  
}
```

JWT Validation at Gateway

Instead of validating JWT in every service:

Centralized Approach

- Gateway validates token
- Only valid requests forwarded

Flow

Client → JWT → Gateway → Validate → Forward

Advantages

- Reduces duplication
- Improves security consistency

Security Insight:

Centralized authentication enforces uniform access control across services.

Centralized Authentication

Concept: Authentication handled at one place (Gateway)

Process

- User logs in via Auth Service
- JWT issued
- Gateway validates JWT for every request

Result

Microservices remain stateless and secure

Spring Security Basics

Spring Security provides:

- Authentication
- Authorization
- Password encoding

Key Concepts

- Security Filter Chain
- Authentication Manager
- Role-based access

Example Configuration


```
http
.csrf().disable()
.authorizeHttpRequests()
.requestMatchers("/auth/**").permitAll()
.anyRequest().authenticated();
```

Integration Flow (Complete System)

- User logs in → Auth Service
- Receives JWT
- Sends request via Gateway
- Gateway validates token
- Request forwarded to respective service

Expected Learning Outcomes

Students will be able to:

- Explain microservices communication patterns
- Implement REST-based service interaction
- Use WebClient for inter-service calls
- Configure API Gateway routes and filters
- Apply centralized authentication using JWT
- Understand Spring Security fundamentals

Deliverables

- API Gateway configured using Spring Cloud Gateway
- Routes defined for all services
- Global filters implemented
- JWT validation at Gateway
- Secure request forwarding

Viva Questions

1. What is microservices communication?
2. Differentiate between synchronous and asynchronous communication.
3. What are the advantages of asynchronous communication?
4. Explain REST communication in microservices.
5. What is RestTemplate and what are its limitations?
6. How does WebClient improve performance?
7. What is an API Gateway and why is it needed?

8. Explain route configuration in API Gateway.
9. What are global filters in Spring Cloud Gateway?
10. How is JWT validated at the gateway level?
11. What is centralized authentication?
12. What are the benefits of using an API Gateway?
13. What is Spring Security and its core functions?
14. How does gateway improve system security?
15. What challenges exist in microservices communication?
16. What is the role of filters in request processing?
17. How does API Gateway reduce system complexity?
18. Explain request flow in a microservices system.
19. Why is WebClient preferred over RestTemplate in a high-traffic microservices environment?
20. What is "Thread Exhaustion," and how does reactive programming solve it?
21. In which scenario would you choose gRPC over REST for inter-service communication?
22. Explain the concept of a "Circuit Breaker" in the context of synchronous calls.
23. What is the primary difference between a Load Balancer and an API Gateway?
24. Explain the role of "Predicates" in Spring Cloud Gateway. Give two examples.
25. How does the lb:// prefix work in the Gateway route configuration?
26. What are "Global Filters," and how do they differ from "Route-specific Filters"?
27. If you need to implement Rate Limiting, where is the best place to do it: the Microservice or the Gateway? Justify.
28. Why is Session-based authentication impractical in a distributed microservices architecture?
29. Describe the three components of a JWT. Which part is used for verification?
30. What is the danger of storing sensitive data (like passwords) in the JWT payload?
31. Explain the step-by-step flow of a request through a Gateway that implements JWT validation.
32. What is the difference between Authentication and Authorization in Spring Security?